САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Малахов Алексей

К3439

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать приложение из ЛР1 и ЛР2 на фреймворк Vue.js. Требования:
- должен быть подключён роутер;
- должна быть реализована работа с внешним API (желательно через axios);
- разумное деление на компоненты;
- использование composable для выделения повторяющегося функционала в отдельные файлы.

Вариант 5: сервис для обмена рецептами и кулинарных блогов.

Ход работы

Проект

За основу взят проект из ДЗ5, созданный командой `npm create vue@latest` с Vue Router и Pinia, к нему подключены axios, Bootstrap и pinia-persists. Моковое API из ЛР2 (JSON Server, json-server-auth и дополнительные проверки `missing.cjs` и `guards.cjs`) перенесено в папку `server/`, картинки и SVG-спрайт в `public/assets/`. Версия на чистом HTML и JS осталась в папке `lab2`.

```
npm run api     # API на http://localhost:3000
npm start       # приложение на http://localhost:8080
npm run build   # production-сборка в dist/
```

Роутер

Все страницы стали маршрутами Vue Router с режимом `createWebHistory`, страницы загружаются лениво через `import()`:

Путь	Имя	Страница
`/`	`home`	главная
`/recipes`	`search`	поиск, фильтры хранятся в query-параметрах
`/recipes/:id`	`recipe`	рецепт, `id` передаётся в компонент через `props`
`/authors/:id`	`author`	кулинар, своя страница перенаправляет в кабинет (`beforeEnter`)
`/profile`	`profile`	личный кабинет, `meta.requiresAuth`
`/login`, `/register`	`login`, `register`	вход и регистрация, `meta.guestOnly`
`/:pathMatch(.*)*`	`not-found`	страница 404

Глобальный guard проверяет `meta`: без входа в кабинет нельзя (переход на вход с параметром `redirect`), вошедшему пользователю незачем открывать вход и регистрацию. После перехода `afterEach` ставит заголовок вкладки из `meta.title`:

```
router.beforeEach((to) => {
  const auth = useAuthStore()
  if (auth.token && !auth.isAuthenticated) auth.logout()

  if (to.meta.requiresAuth && !auth.isAuthenticated) {
    return { name: 'login', query: { redirect: to.fullPath } }
  }
  if (to.meta.guestOnly && auth.isAuthenticated) {
    const redirect = to.query.redirect
    return typeof redirect === 'string' &&
      redirect.startsWith('/') &&
      !redirect.startsWith('//')
      ? redirect
      : { name: 'profile' }
  }
})
```

Ссылки сделаны через `<router-link>`, поэтому переходы происходят без перезагрузки страницы, а кнопки «назад» и «вперёд» браузера работают.

Работа с API через axios

Как в мануале, есть один экземпляр axios и классы для ресурсов (`RecipesApi`, `AuthApi`, `ProfilesApi`, `SocialApi`), которым экземпляр передаётся в конструкторе. Интерцепторы добавляют токен к каждому запросу и в одном месте обрабатывают ошибки. Если сервер ответил 401 (токен недействителен), сессия сбрасывается. Со страницы, которой нужен вход, пользователь попадает на вход с параметром `redirect`, а открытая страница остаётся на месте с сообщением «Сессия истекла, войдите снова». Ошибка сервера превращается в понятный текст:

```
instance.interceptors.request.use((config) => {
  const auth = useAuthStore()
  if (auth.isAuthenticated) config.headers.Authorization = `Bearer ${auth.token}`
  return config
})

instance.interceptors.response.use(
  (response) => response,
  (error) => {
    const auth = useAuthStore()
    if (error.response?.status === 401 && auth.token) {
      auth.logout()
      const current = router.currentRoute.value
      if (current.meta.requiresAuth) {
        router.push({ name: 'login', query: { redirect: current.fullPath } })
      }
    }
    return Promise.reject(toApiError(error))
  },
)
```

Токен и пользователь хранятся в хранилище Pinia `auth`. Благодаря `pinia-persists` оно сохраняется в `localStorage`, пароль там не хранится. Геттер `isAuthenticated` проверяет срок действия JWT по полю `exp`.

Компоненты

Страницы (`views/`) собраны из компонентов (`components/`) и обёрнуты в `BaseLayout`:

Компонент	**Назначение**
`AppHeader`	меню, переключатель темы, вход/выход
`RecipeCard`, `RecipeGrid`	карточка рецепта и сетка с состояниями загрузки, ошибки и пустого списка
`LikeButton`, `SaveButton`, `FollowButton`	кнопки соцфункций
`RecipeFilters`	фильтры поиска (`defineModel`)
`CommentSection`	список комментариев и форма
`BaseModal`, `RecipeFormModal`, `ProfileFormModal`	модальные окна Bootstrap и формы в них
`AuthorPanel`, `AppIcon`, `AppToast`, `LoadingState`, `ErrorAlert`	мелкие переиспользуемые блоки

Компоненты получают данные через props и сообщают о действиях через события (`emit`). Например, `RecipeGrid` выводит `RecipeCard` через `v-for`, а кнопку «Удалить» в кабинете добавляет через именованный слот `actions`, поэтому одна и та же карточка используется на главной, в поиске, на странице автора и в кабинете (Рис. 1, Рис. 5).

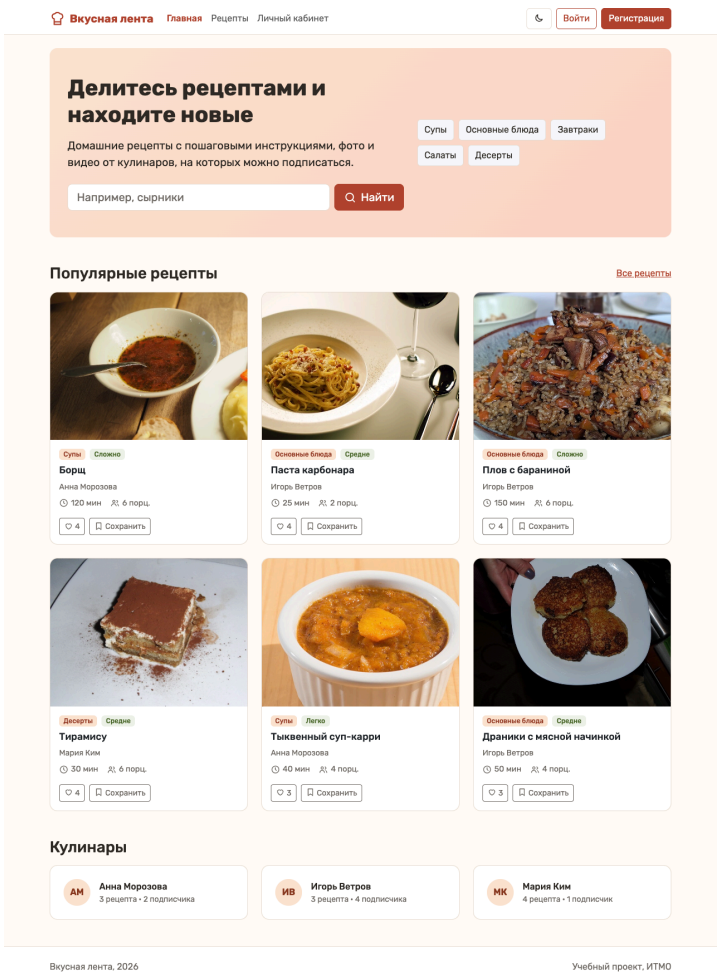


Рис. 1. Главная страница на Vue

Composable

Повторяющаяся логика вынесена в функции `use...` в папке `composables/`:

- `useAsync`: загрузка данных с флагами `loading` и `error`, ответ на устаревший запрос отбрасывается;
- `useLike`, `useSaved`, `useFollow`: лайк, закладка и подписка с запросами к API, блокировкой повторного нажатия и обработкой ответов 404 и 409;
- `useRecipeFilters`: состояние фильтров, двусторонняя синхронизация с URL (неизвестные значения из адреса отбрасываются), запрос к API с задержкой и сортировка;
- `useProfiles`: общий справочник профилей для имён авторов;
- `useFormValidation`: проверка форм, в том числе полей из одних пробелов, `aria-invalid` и фокус на ошибке;
- `useRequireAuth`, `useNotify`, `useTheme`: переход на вход для гостя, всплывающие сообщения, тема.

Например, `useAsync` используется на каждой странице, где загружаются данные:

```
import { ref } from 'vue'

export function useAsync(loader) {
  const data = ref(null)
  const loading = ref(false)
  const error = ref(null)
  let current = 0

  async function run(...args) {
    const request = ++current
    loading.value = true
    error.value = null
    try {
      const result = await loader(...args)
      if (request === current) data.value = result
      return result
    } catch (err) {
      if (request === current) error.value = err
      return null
    } finally {
      if (request === current) loading.value = false
    }
  }

  return { data, loading, error, run }
}
```

`useLike` хранит всю логику лайка, поэтому компонент кнопки остаётся простым:

```
import { computed, ref } from 'vue'

import { likesApi } from '@/api'
import useAuthStore from '@/stores/auth'
import { useNotify } from '@/composables/useNotify'
import { useRequireAuth } from '@/composables/useRequireAuth'

export function useLike(recipe) {
  const auth = useAuthStore()
  const { notify } = useNotify()
  const requireAuth = useRequireAuth()
  const busy = ref(false)

  const myLike = computed(() =>
    recipe.value.likes.find((like) => like.userId === auth.userId),
  )
  const liked = computed(() => auth.isAuthenticated && Boolean(myLike.value))
  const count = computed(() => recipe.value.likes.length)

  async function reload() {
    const { data } = await likesApi.find({ recipeId: recipe.value.id })
    recipe.value.likes = data
  }

  async function toggle() {
    if (!requireAuth() || busy.value) return
    busy.value = true
    try {
      if (myLike.value) {
        await likesApi.remove(myLike.value.id)
        recipe.value.likes = recipe.value.likes.filter(
          (like) => like.id !== myLike.value.id,
        )
      } else {
        const { data } = await likesApi.add({
          recipeId: recipe.value.id,
          userId: auth.userId,
        })
        recipe.value.likes = [...recipe.value.likes, data]
      }
    } catch (error) {
      notify(error.message)
      if ([404, 409].includes(error.status)) await reload()
    } finally {
      busy.value = false
    }
  }

  return { liked, count, busy, toggle }
}
```

```
<template>
  <button
    type="button"
    class="btn btn-outline-secondary btn-like"
    :class="{ active: liked }"
    :disabled="busy"
    :aria-pressed="String(liked)"
    :aria-label="`Нравится «${recipe.title}», отметок: ${count}`"
    @click="toggle"
  >
    <app-icon name="heart" /> <span class="like-count">{{ count }}</span>
  </button>
</template>

<script setup>
import { toRef } from 'vue'

import AppIcon from '@/components/AppIcon.vue'
import { useLike } from '@/composables/useLike'

const props = defineProps({
  recipe: { type: Object, required: true },
})

const { liked, count, busy, toggle } = useLike(toRef(props, 'recipe'))
</script>
```

Страницы

Поиск хранит фильтры в адресе, поэтому ссылку на выборку можно отправить, а после перезагрузки фильтры восстанавливаются (Рис. 2). На странице рецепта работают лайк, закладка, подписка и комментарии (Рис. 3). Личный кабинет показывает вкладки и открывает модальные окна Bootstrap через компонент `BaseModal` (Рис. 4, Рис. 5).

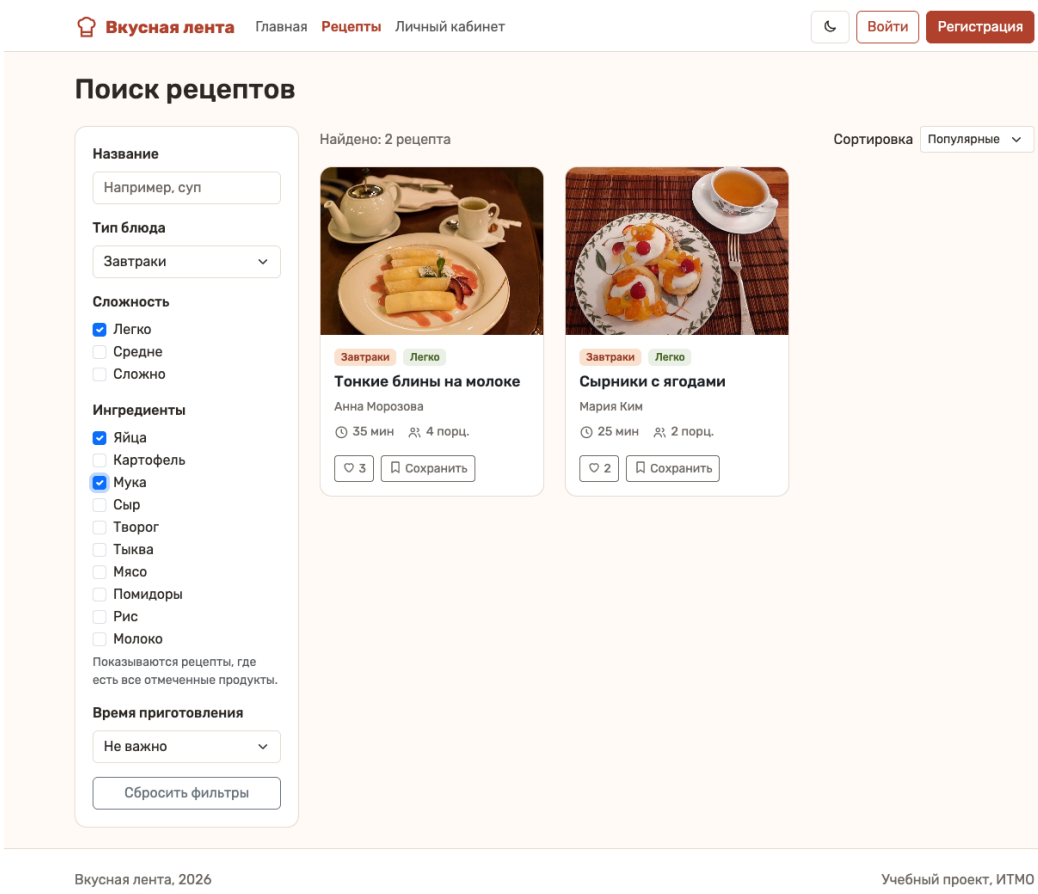


Рис. 2. Поиск: фильтры в URL и запросе к API

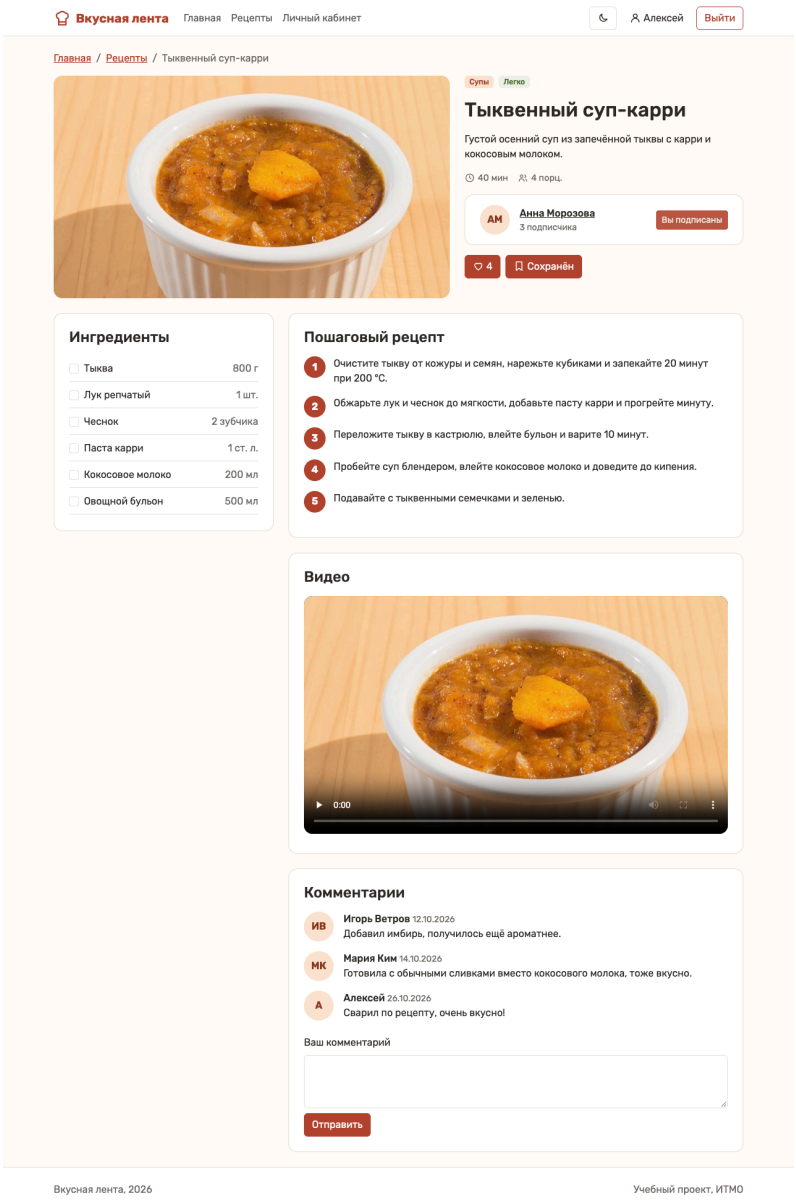


Рис. 3. Страница рецепта после лайка, сохранения, подписки и комментария

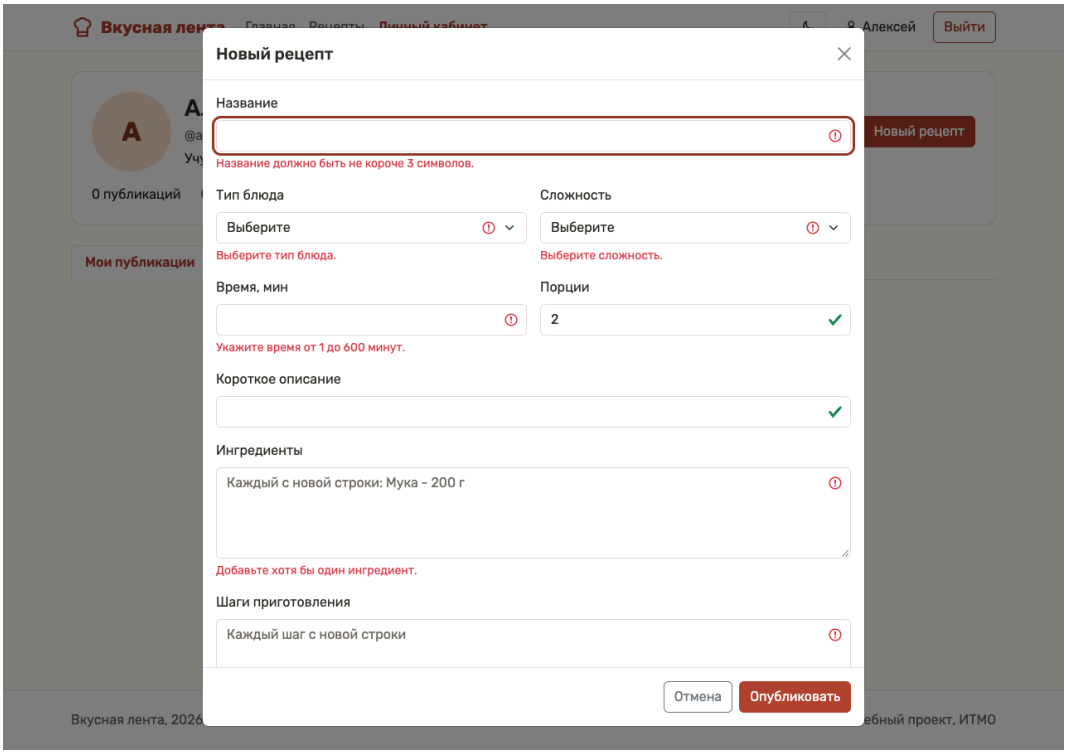


Рис. 4. Модальное окно нового рецепта с проверкой полей

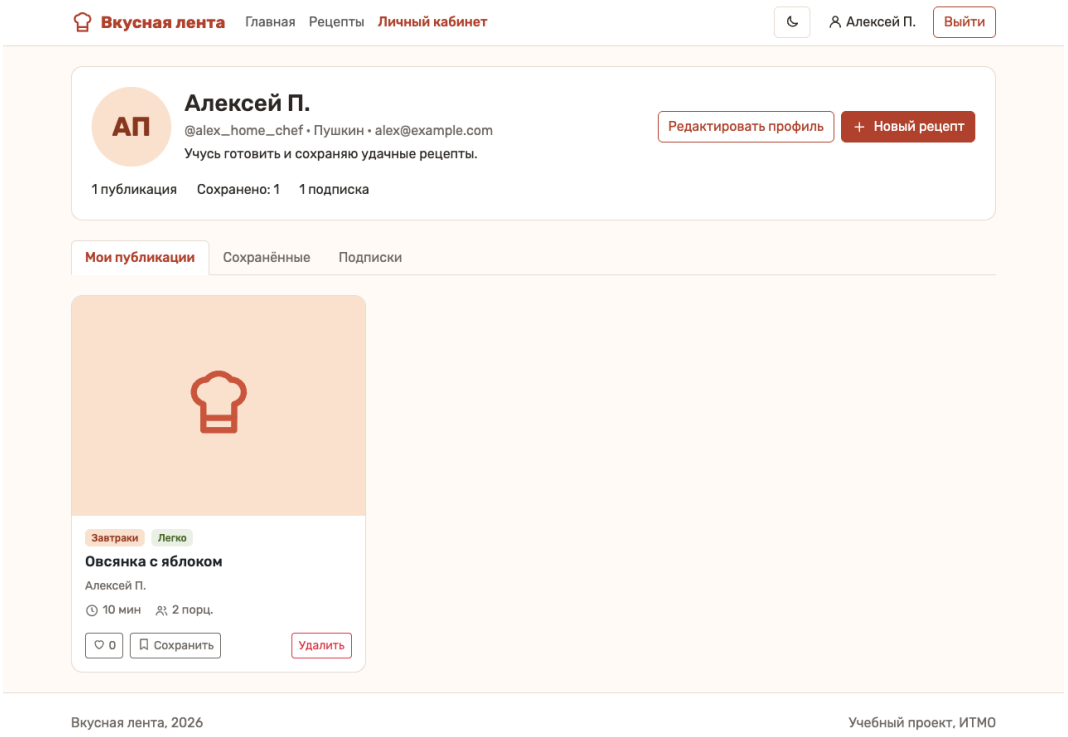


Рис. 5. Личный кабинет после публикации и смены имени

Сохранены доработки из домашних работ: атрибуты доступности (ДЗ2), светлая и тёмная темы (ДЗ3, Рис. 6) и SVG-спрайт, иконки из которого выводит компонент `AppIcon` (ДЗ4). Мобильное меню закрывается после перехода по ссылке (Рис. 7).

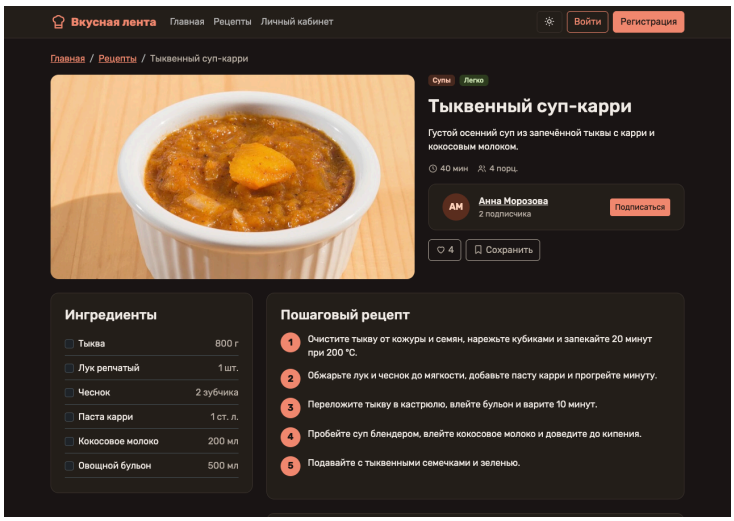


Рис. 6. Тёмная тема

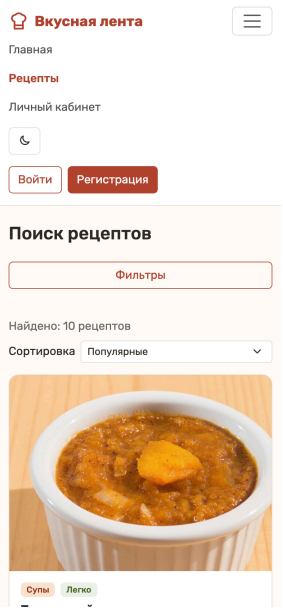


Рис. 7. Меню на ширине 390 px

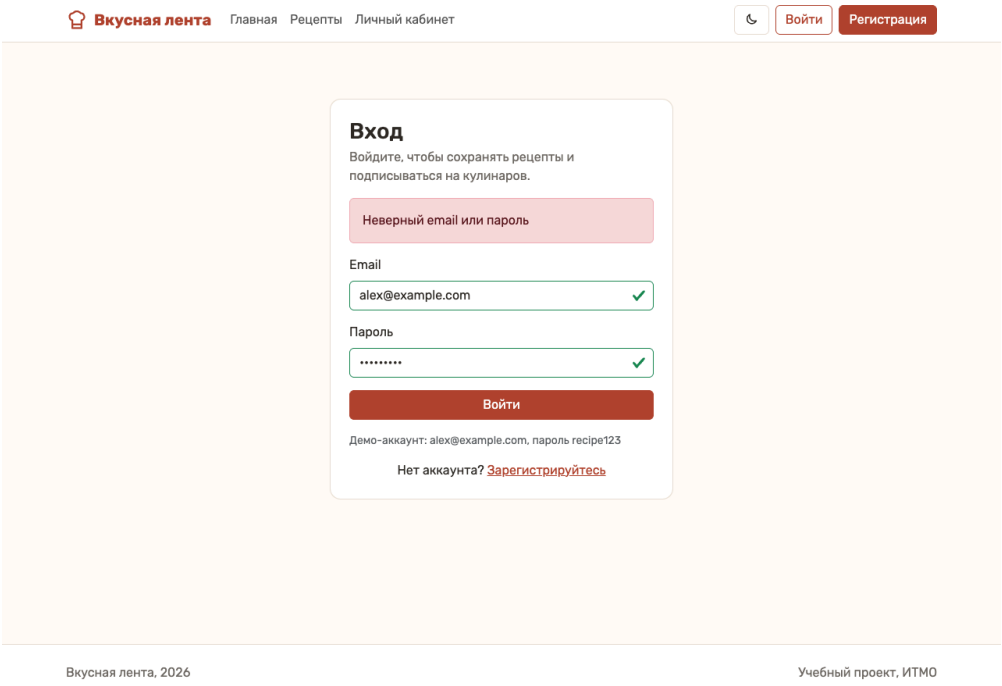


Рис. 8. Вход: ответ сервера на неверный пароль

Проверка

Сценарии проходились в Google Chrome под macOS (через Playwright) дважды: на dev-сервере (`npm start`) и на production-сборке (`npm run build`, `vite preview`). В обоих случаях прошли все 20 сценариев: данные главной из API, переходы через `router-link` без перезагрузки страницы, фильтры в URL и после перезагрузки, сброс фильтров ссылкой в шапке и некорректные значения в адресе, назад и вперёд в истории, вход с `redirect` и неверный пароль (Рис. 8), лайк, закладка, подписка и комментарий с проверкой после перезагрузки, вкладки и модальные окна кабинета, удаление рецепта двойным кликом

(уходит один запрос) и отписка, guards `requiresAuth`, `guestOnly` и `beforeEnter`, несуществующий рецепт и маршрут 404, регистрация, испорченный токен на открытой странице и в кабинете, недоступное API, темы, мобильная версия. Ошибок JavaScript не было. В production-сборке прямой заход на `/recipes/1`, `/authors/2` и `/profile` открывает приложение (отвечает `index.html`).

Lighthouse (раздел Accessibility) на production-сборке в светлой теме даёт 100 баллов на главной, в поиске, на странице рецепта, на странице кулинара и на входе (Рис. 9). `npx eslint .`, `npx oxlint .` и `npx prettier --check src/` проходят без замечаний. Сборка `npm run build`:

```
dist/assets/SearchView-DluYKngO.js                  6.35 kB │ gzip:  2.83 kB
dist/assets/RecipeView-C-neGUc2.js                  7.27 kB │ gzip:  3.36 kB
dist/assets/ProfileView-N6_w5Qf8.js                14.26 kB │ gzip:  5.06 kB
dist/assets/_plugin-vue_export-helper-BaxhD9_W.js  127.34 kB │ gzip: 49.83 kB
dist/assets/index-Dncc1u98.js                      257.02 kB │ gzip: 88.26 kB
✓ built in 214ms
```

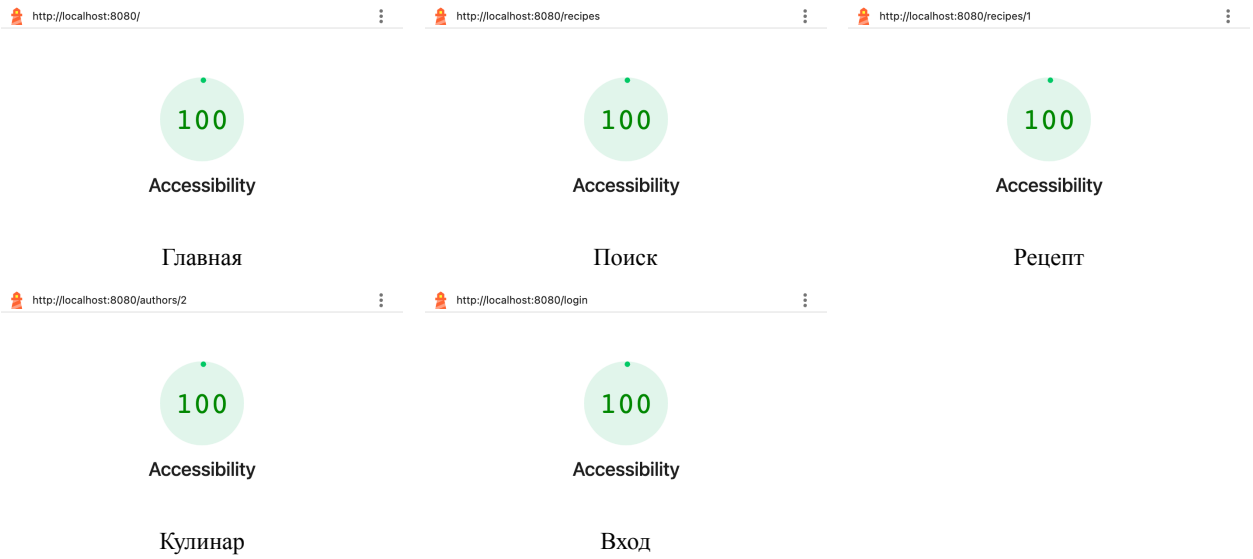


Рис. 9. Lighthouse на production-сборке

Вывод

Сайт рецептов перенесён на Vue.js как одностраничное приложение. Подключён Vue Router с ленивой загрузкой страниц, параметрами маршрутов, `meta` и navigation guards. Работа с API сделана через axios: общий экземпляр с интерцепторами для токена и ошибок и классы для ресурсов. Интерфейс разделён на страницы, лейаут и переиспользуемые компоненты с props, событиями, слотами и `defineModel`. Повторяющаяся логика (загрузка данных, лайки, закладки, подписки, фильтры, валидация, тема) вынесена в composable. Состояние авторизации хранится в Pinia и сохраняется между перезагрузками.